



**UNIVERSIDAD DE DISEÑO,
INNOVACIÓN Y TECNOLOGÍA**

**Grado en Diseño y Desarrollo
de Videojuegos
(Curso 2025 - 2026)**

Entrega: Fábrica (28/4/2026)

Ramón Zabara Morales

Índice

• Descripción general	3
• Diagrama de clases	4
◦ Factory Line	4
◦ Factory Manager	5
◦ Factory Widget	6
◦ Emergency Panel Widget	7
◦ Game Over Widget	8
• Diario de desarrollo	9
• Anexos	9
• Bibliografía	10

1 Descripción general

Esta practica consiste en implementar una fábrica automatizada con tres líneas de producción que gestionan eficiencia y recursos mediante multithreading en Unreal Engine 5 con C++. Las actualizaciones de las líneas se comunican con delegados y eventos a un gestor central que actualiza la UI, que muestra indicadores visuales y alertas sobre el estado de las líneas. Cuando una línea entra en estado crítico, se activa un minijuego de emergencia donde el usuario debe intervenir para restablecer la linea y evitar fallos.



2 Diagrama de clases

2.1 Factory Line

Para implementar la simulación de la fábrica, se desarrolló la clase **FactoryLine** que representa cada línea y gestiona su comportamiento en tiempo real mediante multithreading, delegados y eventos.

Las variables:

- **state** → Estado actual de la línea (Operative, Warning o Critical).
- **efficiency** → Nivel de eficiencia de producción (0 - 100).
- **resources** → Nivel de recursos disponibles (0 - 100).
- **maxValue** → Valor máximo permitido para los parámetros.
- **warningThreshold** → Umbral para estado de advertencia.
- **criticalThreshold** → Umbral para estado crítico.
- **maxDecreasingValue** → Valor máximo de la variación aleatoria.
- **running** → Controla si el thread sigue activo.
- **active** → Indica si la línea continúa funcionando tras un fallo.
- **randomStream** → Generador de valores aleatorios para la línea.

Los delegados:

- **OnProductionChanged** → Notifica cambios de valores y estado.
- **OnProductionSave** → Se lanza al salvar una línea crítica.
- **OnProductionFail** → Se ejecuta cuando la línea falla.
- **OnProductionReset** → Notifica el reinicio completo de la línea.

Las funciones:

- **StartSimulation()** → Inicia un thread que actualiza la eficiencia y los recursos cada segundo.
- **EvaluateState()** → Comprueba los valores actuales y cambia el estado según los umbrales establecidos.
- **SaveLine()** → Restaura la línea a valores seguros tras una emergencia.

2 Diagrama de clases

2.1 Factory Line

- **FailLine()** → Desactiva la línea al no intervenir a tiempo.
- **ResetLine()** → Reinicia completamente la línea tras un Game Over.
- **BeginPlay()** / **EndPlay()** → Inician y finalizan la simulación al comenzar o acabar el juego o al spawnear o despawnear al actor.

2.2 Factory Manager

Para coordinar el funcionamiento general del juego, se desarrolló la clase **FactoryManager**, encargada de gestionar todas las líneas de producción, recibir sus eventos, actualizar la UI y controlar las situaciones de emergencia y el reinicio del juego.

Las variables:

- **foundLines** → Array con las líneas de producción en la escena.
- **emergencyActive** → Array que indica si una línea tiene una emergencia activa.
- **failedLines** → Array que guarda qué líneas han fallado.
- **score** → Puntuación obtenida al salvar líneas críticas.
- **factoryWidgetClass** / **emergencyPanelWidgetClass** / **gameOverWidgetClass** → Clases de los distintos widgets.
- **factoryWidget** → Referencia al widget creado durante la ejecución.

Los delegados:

- **OnFactoryLineUpdated** → Envía a la UI los nuevos valores de cada línea.
- **OnResetGame** → Notifica el reinicio global de la partida.

2 Diagrama de clases

2.2 Factory Manager

Las funciones:

- **BeginPlay()** → Busca todas las líneas de producción, aplica los delegados y crea la interfaz principal del juego.
- **HandleProductionStateChanged()** → Recibe cambios de una línea, actualiza la UI y activa el panel de emergencia si entra en estado crítico.
- **SaveEmergency()** → Marca la emergencia como resuelta y aumenta la puntuación.
- **FailEmergency()** → Registra una línea fallida y comprueba si todas han parado para mostrar Game Over.
- **ResetEmergency()** → Reinicia los arrays y puntuación y envía la orden de reset a todas las líneas.
- **Tick()** → Actualización en cada frame.

2.3 Factory Widget

Para representar visualmente el estado de la fábrica, se desarrolló la clase **FactoryWidget**, encargada de actualizar la interfaz gráfica mostrando en tiempo real la información de cada línea de producción mediante barras de progreso e indicadores de color.

Las variables:

- **efficiencyLine1, efficiencyLine2, efficiencyLine3** → Barras de progreso de eficiencia para cada línea.
- **resourcesLine1, resourcesLine2, resourcesLine3** → Barras de progreso de recursos para cada línea.
- **stateLine1, stateLine2, stateLine3** → Imágenes que indican visualmente el estado de cada línea.

2 Diagrama de clases

2.3 Factory Widget

Las funciones:

- **HandleFactoryLineUpdated()** → Recibe los datos enviados por el gestor central y actualiza visualmente la línea correspondiente.
- **SetEfficiencyValue()** → Modifica la barra de eficiencia de una línea concreta.
- **SetResourceValue()** → Modifica la barra de recursos de una línea concreta.
- **SetStateColor()** → Cambia el color del indicador según el estado actual (verde (*Operative*), amarillo (*Warning*) o rojo (*Critical*)).

2.4 Emergency Panel Widget

Para gestionar las situaciones críticas de una línea de producción, se desarrolló la clase **EmergencyPanelWidget**, encargada de mostrar un panel de emergencia interactivo donde el jugador debe reaccionar rápidamente para evitar el fallo de la línea.

Las variables:

- **saveButton** → Botón que permite salvar la línea en emergencia.
- **targetLine** → Referencia a la línea de producción afectada.
- **emergencyTimer** → Temporizador que limita el tiempo de respuesta del jugador para salvar la línea.

Los delegados:

- **OnLineReset** → Notifica que la línea ha sido recuperada correctamente.
- **OnLineFailed** → Notifica que la línea ha fallado al no intervenir a tiempo.

2 Diagrama de clases

2.4 Emergency Panel Widget

Las funciones:

- **NativeConstruct()** → Vincula el botón de salvar línea con su evento al crear el widget.
- **Init()** → Inicializa el panel con la línea afectada y activa el temporizador de la emergencia.
- **OnSavePressed()** → Se ejecuta al pulsar el botón a tiempo, restableciendo la línea y cerrando el panel.
- **OnEmergencyFailed()** → Se activa cuando el tiempo se agota, marcando la línea como fallida y cerrando el panel.

2.5 Game Over Widget

Para gestionar el final de la partida, se desarrolló la clase **GameOverWidget**, encargada de mostrar la puntuación final obtenida por el jugador y permitir reiniciar el juego.

Las variables:

- **scoreText** → Texto que muestra la puntuación final obtenida.
- **resetButton** → Botón para reiniciar la partida.

Los delegados:

- **OnResetPressed** → Notifica que se ha pulsado el botón de reinicio.

Las funciones:

- **NativeConstruct()** → Vincula el botón de reinicio con su evento al crear el widget.
- **Init()** → Inicializa el panel mostrando la puntuación final conseguida.
- **ResetButtonPressed()** → Se ejecuta al pulsar el botón, lanza el evento de reinicio y cierra el panel.

3 Diario de desarrollo

Durante el desarrollo de la práctica han habido grandes complicaciones dado que en mi PC tenía que cerrar todos los programas abiertos cada vez que quería compilar el proyecto debido a la falta de RAM, y tener que ir abriendo y cerrando la app de Visual Studio retrasaba mucho la implementación del proyecto. Sobre todo fue duro a la hora de implementar los delegates porque daban bastantes errores y por cada compilación por lo antes mencionado se tardaba mucho en comprobar y debuguear.

4 Anexos



4 Anexos



5 Bibliografía

- Documento explicativo sobre los delegators de C++ en Unreal de Luis Rubio.



**UNIVERSIDAD DE DISEÑO,
INNOVACIÓN Y TECNOLOGÍA**

**Grado en Diseño y Desarrollo de
Videojuegos**

Fábrica

Ramón Zabara Morales